

Shell Subsystem

Zephyr RTOS Course





Objectives



- Understand the **Shell subsystem** architecture
- Configure shell **backends** (UART, RTT, Telnet, USB)
- Create **static**, **dynamic**, and **dictionary** commands
- Use **built-in commands** and shell features
- Integrate shell with the **logging** subsystem



Quick Recap



- **Zephyr's Device Driver Model**
- Driver directory layout.
- Standard driver APIs and subsystem-specific APIs.
- Writing a custom driver and exposing it through Device Tree.



Format



- Lecture with optional **live demo**
- Homework after each session (checked by us)
- Recordings will be shared
- Questions addressed at the end in a **Q&A session**



What Is the Shell?



Shell Overview



A **Unix-like interactive CLI** running on your embedded device:

- Support for **multiple instances** (UART + Telnet simultaneously)
- **Tab completion** and command **history**
- **Static and dynamic** command registration
- Built-in commands: `clear`, `history`, `shell`, `log`
- Integration with the **Logging** subsystem
- ANSI escape codes for cursor control and colors



Enabling the Shell



Minimal `prj.conf`:

```
CONFIG_SHELL=y
```

The shell starts automatically on the default backend (usually UART).

Additional useful options:

```
CONFIG_SHELL_CMDS=y  
CONFIG_SHELL_TAB=y  
CONFIG_SHELL_TAB_AUTOCOMPLETION=y  
CONFIG_SHELL_HISTORY=y  
CONFIG_SHELL_LOG_BACKEND=y
```



Minimal Shell Mode



For **resource-constrained** devices:

```
CONFIG_SHELL_MINIMAL=y
```

- Disables most features by default
- Selectively re-enable only what you need
- Significant **RAM and flash** savings



Shell Backends



Available Backends



Backend	Kconfig	Use Case
UART	<code>CONFIG_SHELL_BACKEND_SERIAL</code>	Default, local debug
USB CDC ACM	Snippet: <code>cdc-acm-console</code>	USB serial console
Telnet	<code>CONFIG_SHELL_BACKEND_TELNET</code>	Remote access over IP
Segger RTT	<code>CONFIG_SHELL_BACKEND_RTT</code>	Debug probe (no UART)
BLE (NUS)	Snippet: <code>nus-console</code>	Wireless shell over BLE
DUMMY	<code>CONFIG_SHELL_BACKEND_DUMMY</code>	Testing / programmatic



UART Backend (Default)



```
CONFIG_SHELL_BACKEND_SERIAL=y
```

Connect with any serial terminal (minicom, PuTTY, screen):

```
minicom -D /dev/ttyACM0 -b 115200
```



USB CDC ACM Backend



Add the snippet to your build:

```
west build -S cdc-acm-console [...]
```

- Turns USB port into a **virtual serial port**
- No extra UART wires needed
- Device appears as `/dev/ttyACM*` on Linux



Telnet Backend



```
CONFIG_SHELL_BACKEND_TELNET=y  
CONFIG_NETWORKING=y
```

Connect remotely:

```
telnet 192.168.1.100 4242
```

- Great for **networked devices** — debug without physical access
- Optional: `CONFIG_SHELL_TELNET_SUPPORT_COMMAND=y` for character-at-a-time mode



Segger RTT Backend



```
CONFIG_USE_SEGGER_RTT=y  
CONFIG_SHELL_BACKEND_RTT=y  
CONFIG_SHELL_BACKEND_SERIAL=n    # Disable UART if not needed
```

Connect via J-Link:

```
west rtt
```



Shell Commands



Command Tree Structure



Commands are organized in a **tree hierarchy**:

- **Root command** (level 0) — top-level entry point
- **Subcommands** (level > 0) — children of a root command
- Each node can have a **handler** and/or **children**

```
demo          ← root command (level 0)
├── ping       ← subcommand (level 1)
└── params    ← subcommand (level 1)
```




Command Handler Signature



Every command handler follows the same pattern:

```
static int cmd_handler(const struct shell *sh, size_t argc,  
                      char **argv)  
{  
    ARG_UNUSED(argc);  
    ARG_UNUSED(argv);  
    shell_print(sh, "Hello from the shell!");  
    return 0; // 0 = success  
}
```



Shell Print Functions



```
// Simple text output
shell_print(sh, "Print simple text.");

// Colored/categorized output
shell_info(sh, "Informational message");    // default color
shell_warn(sh, "Warning message");          // yellow
shell_error(sh, "Error message");           // red

// Formatted output with explicit color
shell_fprintf(sh, SHELL_INFO0, "Value: %d\n", 42);
```



Creating Static Commands



Registering a Root Command



```
#include <zephyr/shell/shell.h>
static int cmd_version(const struct shell *sh, size_t argc, char **argv) {
    shell_print(sh, "App version: 1.0.0");
    return 0;
}
/* Register root command "version" */
SHELL_CMD_REGISTER(version, NULL, "Show app version", cmd_version);
```

In the shell:

```
uart:~$ version
App version: 1.0.0
```



```
static int cmd_demo_salute(const struct shell *sh, size_t argc, char **argv) {
    shell_print(sh, "Good Afternoon");
    return 0;
}

static int cmd_demo_parse(const struct shell *sh, size_t argc, char **argv) {
    for (int i = 0; i < argc; i++) {
        shell_print(sh, "  argv[%d] = %s", i, argv[i]);
    }
    return 0;
}
```



Static Subcommands — Registration



```
/* Create subcommands array for "demo" */
SHELL_STATIC_SUBCMD_SET_CREATE(sub_demo,
    SHELL_CMD(salute, NULL, "Print params command.", cmd_demo_salute),
    SHELL_CMD(parse, NULL, "Salute command.", cmd_demo_parse),
    SHELL_SUBCMD_SET_END
);
/* Register root command "demo" with subcommands */
SHELL_CMD_REGISTER(demo, &sub_demo, "Demo commands", NULL);
```



Static Subcommands — At The Shell



```
uart:~$ demo salute
Good Afternoon
uart:~$ demo parse hello world
  argv[0] = hello
  argv[1] = world
```



Dynamic Commands



Subcommands generated **at runtime** — the list can change:

```
static char dynamic_cmd_buffer[10][50];
static uint8_t dynamic_cmd_cnt;
static void dynamic_cmd_get(size_t idx, struct shell_static_entry *entry) {
    if (idx < dynamic_cmd_cnt) {
        entry->syntax  = dynamic_cmd_buffer[idx];
        entry->handler = NULL;
        entry->subcmd  = NULL;
        entry->help    = "Dynamic command.";
    } else {
        entry->syntax = NULL;
    }
}
SHELL_DYNAMIC_CMD_CREATE(m_sub_dynamic, dynamic_cmd_get);
```



Dynamic Commands — Registration



```
SHELL_STATIC_SUBCMD_SET_CREATE(m_sub_dynamic_cmds,  
    SHELL_CMD(add, NULL,  
        "Add a dynamic command.",    cmd_dynamic_add),  
    SHELL_CMD(execute, &m_sub_dynamic,  
        "Execute a command.",        cmd_dynamic_execute),  
    SHELL_CMD(remove, &m_sub_dynamic,  
        "Remove a command.",         cmd_dynamic_remove),  
    SHELL_CMD(show, NULL,  
        "Show all commands.",        cmd_dynamic_show),  
    SHELL_SUBCMD_SET_END  
);  
SHELL_CMD_REGISTER(dynamic, &m_sub_dynamic_cmds,  
    "Dynamic command demo.", NULL);
```



Dynamic Commands — At The Shell



```
uart:~$ dynamic add hello
uart:~$ dynamic add world
uart:~$ dynamic execute <Tab>
hello world
```



For You To Dig In



Read The Docs!



- Arguments automatic validation with `SHELL_CMD_ARG`
- Dictionary commands for mapping strings to data `SHELL_SUBCMD_DICT_SET_CREATE`
- Parent Command Access
- Wildcards
- `sys_getopt` — Option Parsing
- Obscured Input



Built-in Commands



Built-in Command Groups



Many Zephyr subsystems provide **built-in shell commands**:

Subsystem	Enable Kconfig
GPIO	<code>CONFIG_GPIO_SHELL=y</code>
I2C	<code>CONFIG_I2C_SHELL=y</code>
Sensor	<code>CONFIG_SENSOR_SHELL=y</code>
Flash	<code>CONFIG_FLASH_SHELL=y</code>
ADC	<code>CONFIG_ADC_SHELL=y</code>
File System	<code>CONFIG_FILE_SYSTEM_SHELL=y</code>
Kernel	<code>CONFIG_KERNEL_SHELL=y</code>



Demo: Custom Shell Commands



Demo: Continue L06 Sandbox



We reuse the previous lecture project and expose the custom driver through shell.

Path:

```
/home/mariano/zephyrproject/apps/sandbox/app
```

Build and flash:

```
cd /home/mariano/zephyrproject/apps/sandbox/app  
west build -b nucleo_f429zi  
west flash
```



Demo: Shell Command Registration



```
SHELL_STATIC_SUBCMD_SET_CREATE(  
    sub_motor,  
    SHELL_CMD(on, NULL, "Turn motor ON", cmd_motor_on),  
    SHELL_CMD(off, NULL, "Turn motor OFF", cmd_motor_off),  
    SHELL_CMD(status, NULL, "Show GPIO and driver state", cmd_motor_status),  
    SHELL_SUBCMD_SET_END);  
  
SHELL_CMD_REGISTER(motor, &sub_motor,  
    "Custom driver test commands", NULL);
```



Demo: Live Test Sequence



```
uart:~$ motor status
GPIO: port=GPIOB pin=7, state=0FF
uart:~$ motor on
Motor state set to ON
uart:~$ motor -h
motor - Custom driver test commands
```



Best Practices



Shell Best Practices



- ✓ Use `shell_print` / `shell_warn` / `shell_error` — **never** `printf`
- ✓ Keep handlers **short** — offload heavy work to other threads
- ✓ Use `SHELL_CMD_ARG` to validate argument count
- ✓ Add **help strings** to every command and subcommand
- ✓ Use `CONFIG_SHELL_MINIMAL` for production builds
- ✗ Don't call shell print functions from **ISR context**
- ✗ Don't block the shell thread for extended periods
- ✗ Don't forget `SHELL_SUBCMD_SET_END` in subcommand arrays



Homework



Home Task 1



Using the sensor driver from L06 Task 1 (LED-based sensor driver):

- Add a `sensor` root shell command with subcommands:
 - `fetch` – calls `sensor_sample_fetch()`
 - `read` – calls `sensor_channel_get()` and prints the result
 - `info` – prints the device name and ready state
- Commit and tag it **I7-task1**.



Home Task 2



Using the custom extension API from L06 Task 2:

- Expose the extension API as a shell subcommand (e.g., `sensor set <value>`)
- The command must validate its argument: print an error via `shell_error` if missing or out of range
- Use `SHELL_CMD_ARG` to enforce argument count
- Commit and tag it **l7-task2**.

Thank You!

Questions?





Resources